

GITHUB ACTIONS



PRODUCTION HANDBOOK

From Your First Workflow to
Production CI/CD Automation



CODE
PUSH



BUILD
& TEST



PACKAGE



DEPLOY



MONITOR



SUCCEED



AUTOMATE
WORKFLOWS



SECURE
PIPELINES



DEPLOY
TO ANY CLOUD



INTEGRATE
WITH YOUR TOOLS



PRODUCTION
BEST PRACTICES

- ✓ PRACTICAL EXAMPLES
- ✓ REAL-WORLD USE CASES
- ✓ SECURITY BEST PRACTICES
- ✓ TROUBLESHOOTING GUIDE
- ✓ PRODUCTION-READY WORKFLOWS

Commit
Automate
Deliver
Repeat

BUILD SKILLS. AUTOMATE FASTER.
DEPLOY WITH CONFIDENCE.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



GITHUB ACTIONS

EXPLAINED, WHY ENGINEERS USE IT

1 WHAT GITHUB ACTIONS IS

- GitHub Actions is a CI/CD automation platform built into GitHub.
- It lets you automate tasks directly in your repository using YAML files called workflows.
- You can build, test, deploy, and manage your applications without leaving GitHub.
- It integrates with thousands of tools and cloud platforms.



2 THE PROBLEM CI/CD SOLVES

- Manual processes are slow, error-prone, and not consistent.
- Teams often forget steps, leading to failed deployments.
- As your application grows, manual deployment becomes difficult to manage.
- CI/CD automation solves this by running the same steps automatically, every time.



**FASTER
SAFER
CONSISTENT
SCALABLE**

3 MANUAL DEPLOYMENT vs AUTOMATED WORKFLOW

MANUAL DEPLOYMENT

- ✗ Run commands manually
- ✗ Easy to make mistakes
- ✗ Not consistent
- ✗ Time consuming
- ✗ Difficult to scale
- ✗ Harder to track changes

AUTOMATED WORKFLOW

- ✓ Runs automatically
- ✓ Consistent every time
- ✓ Fewer human errors
- ✓ Faster delivery
- ✓ Scales easily
- ✓ Fully tracked and auditable

4 EVENTS → WORKFLOWS → JOBS → STEPS → ACTIONS



EVENTS

A trigger starts the workflow (e.g. push, pull request).



WORKFLOWS

A YAML file defines what happens.



JOBS

A set of steps that run on the same runner.



STEPS

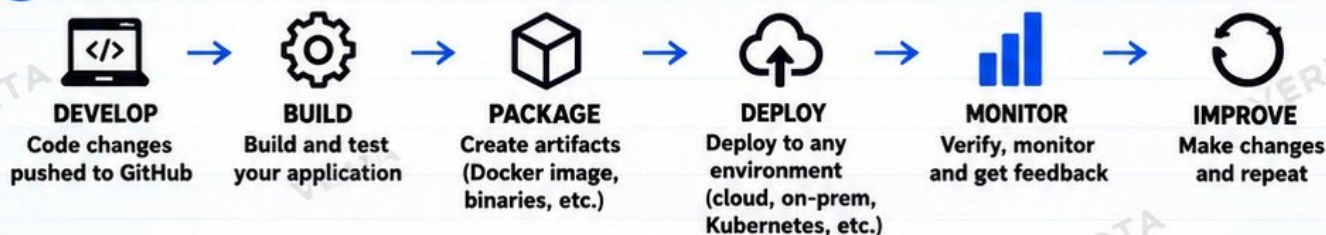
Individual tasks that run in order.



ACTIONS

Reusable components that perform the task (e.g. checkout, setup tools).

5 WHERE GITHUB ACTIONS FITS INTO DEVOPS



GitHub Actions connects your code to your tools, infrastructure, and environments.

6 JUNIOR ENGINEER TIP



Automation does not remove the need to understand the deployment process. You still need to know how your application is built, deployed, and runs in production. GitHub Actions makes you faster, not a replacement for your engineering knowledge.



**AUTOMATE
SMARTER
NOT BLINDLY**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD • LEARN • AUTOMATE • GROW

Same Skills
A Brighter You



GITHUB ACTIONS

ARCHITECTURE AND MENTAL MODEL

Understand how all the pieces work together from a GitHub event to running automation.

1 REPOSITORY



- Your code lives in a GitHub repository.
- GitHub Actions runs in the context of this repository.
- Workflows are stored in `.github/workflows/`.

2 `.github/workflows/`



- A special directory in your repository.
- Contains YAML files that define your workflows.
- Example: `.github/workflows/ci.yml`

3 WORKFLOW



- A YAML file that defines automation.
- Contains one or more jobs.
- Triggered by events such as push, pull request, or a schedule.

4 EVENT



- A trigger that starts a workflow.
- Examples: push, pull_request, workflow_dispatch, schedule.
- You can filter events by branch, path, or tags.

5 JOB



- A set of steps that run on the same runner.
- Jobs can run in parallel or sequentially.
- Use `needs` to define dependencies between jobs.

6 STEP



- An individual task within a job.
- Steps run in order.
- A step can run a shell command or use an action.

7 ACTION



- A reusable component that performs a task.
- Can be built by GitHub, the community, or you (custom actions).
- Examples: actions/checkout, setup-node, docker/login-action.

8 RUNNER



- A machine that runs your jobs.
- GitHub-hosted runners (provided by GitHub).
- Self-hosted runners (provided and managed by you).

9 GITHUB-HOSTED RUNNERS



- Provided and maintained by GitHub.
- Available on Linux (Ubuntu), Windows, and macOS.
- Clean, fresh environment for each run.
- No infrastructure to manage.
- Ideal for most use cases.

10 SELF-HOSTED RUNNERS



- Hosted and managed by you.
- Can run on your own servers or cloud.
- Useful for custom environments, private networks, or special tools.
- Requires maintenance, security updates, and monitoring.
- More control, but more responsibility.

11 GITHUB ACTIONS FLOW (ARCHITECTURE DIAGRAM)



JUNIOR ENGINEER TIP

Automation does not remove the need to understand the deployment process. You still need to know what your application does, how it is built, and how it runs in production. GitHub Actions makes you faster, but it is not a replacement for your engineering knowledge.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD YOUR FIRST WORKFLOW

Creating a simple workflow file and understanding each part

1 CREATE THE WORKFLOW FILE

- In your repository, create a directory `.github/workflows/`
- Inside it, create a file named `ci.yml`
- This file contains your workflow configuration in YAML format.

```

your-repo/
├── .github/
│   └── workflows/
│       └── ci.yml
    
```

2 EXAMPLE: A SIMPLE WORKFLOW

```

name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Run a simple command
        run: echo "Hello from GitHub Actions!"
    
```



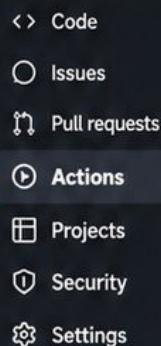
This workflow runs on push or pull request to the main branch, checks out the code, and runs a simple command.

3 UNDERSTANDING EACH PART

name	The name of your workflow (displayed in the Actions tab).
on	Defines the event that triggers the workflow (push, pull_request, etc.).
jobs	A collection of steps that run on the same runner.
runs-on	Specifies the type of runner (e.g. ubuntu-latest).
steps	A list of tasks that run in order.
uses	Runs a pre-built action from the GitHub Actions marketplace.
run	Runs a shell command.
actions/checkout	An official action that checks out your repository code into the runner so you can work with it.

4 READING YOUR FIRST WORKFLOW RUN

- 1 Go to your repository on GitHub.
- 2 Click the Actions tab.
- 3 Select the workflow run.
- 4 You will see:
 - Workflow name
 - Triggered event (e.g. push)
 - Jobs and their status (success or failure)
 - Each step with logs
 - Total run time



5 PRACTICE LAB

- ✓ Create a new repository (or use an existing one).
- ✓ Add a `.github/workflows/ci.yml` file with the example above.
- ✓ Push the file to the main branch.
- ✓ Go to the Actions tab and confirm the workflow runs successfully.
- ✓ Modify the run command to print a different message and run it again.



JUNIOR ENGINEER TIP

Automation does not remove the need to understand the deployment process. You should know what the workflow does, how it builds, tests, and deploys your application, and how to troubleshoot when something fails.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

WORKFLOW TRIGGERS AND EVENTS

Control when your workflows run

1 PUSH



- Runs when code is pushed to a branch.
- Commonly used for CI (build and test).
- Can be filtered by branch or path.

```
on:
  push:
    branches: [ main ]
```

2 PULL_REQUEST



- Runs when a pull request is opened, updated, or closed.
- Useful for testing changes before merging.
- Can be filtered by branch, path, or PR type.

```
on:
  pull_request:
    branches: [ main ]
    types: [opened, synchronize, reopened]
```

3 WORKFLOW_DISPATCH



- Allows manual triggering of a workflow.
- You can define input parameters.
- Useful for running deployments or on-demand tasks.

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment to deploy'
        required: true
        type: choice
        options: [dev, staging, production]
```

4 SCHEDULE



- Runs at a specific time using cron syntax.
- Useful for backups, updates, or maintenance tasks.
- Runs on the default branch only.

```
on:
  schedule:
    - cron: '0 2 * * *' # Every day at 2 AM
```

5 BRANCH FILTERS



- Run workflows only on specific branches.
- Use branches or branches-ignore.

```
on:
  push:
    branches: [ main, develop ]
    branches-ignore: [ 'feature/*' ]
```

6 PATH FILTERS



- Run workflows only when specific files change.
- Use paths or paths-ignore.

```
on:
  push:
    paths: [ 'src/**', 'Dockerfile' ]
    paths-ignore: [ 'docs/**' ]
```

7 TAGS



- Run workflows when a tag is pushed.
- Useful for releases.

```
on:
  push:
    tags: [ 'v*' ]
```

8 MULTIPLE EVENTS



- You can define multiple events in one workflow.
- Useful for handling different use cases.

```
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
```

9 MANUAL RUNS



- Triggered using workflow_dispatch.
- Useful for deployments, testing, or one-time tasks.
- Can accept input parameters.

10 AVOIDING UNNECESSARY PIPELINE EXECUTION



- Use branch and path filters.
- Ignore documentation or non-code changes.
- Choose the right events for the right job.
- This saves time, reduces costs, and keeps your pipelines focused.



JUNIOR ENGINEER TIP

Triggers control when your workflow runs, but you still need to understand what the workflow does. Use the right triggers to run only what is necessary.

JOBS, STEPS, ACTIONS, AND EXECUTION ORDER

Understand how work is executed in a workflow

1 HOW JOBS EXECUTE



- A workflow can have one or more jobs.
- By default, jobs run in parallel.
- Use **needs** to control the order between jobs.
- Each job runs on a separate runner.

2 SEQUENTIAL STEPS



- Steps within a job run sequentially (one after another).
- If a step fails, the remaining steps in that job are skipped (unless **continue-on-error** is used).

3 PARALLEL JOBS



- Jobs run in parallel by default.
- Each job uses its own runner.
- Useful for faster execution (e.g. test on multiple versions).

4 NEEDS



- The **needs** keyword makes a job wait for another job to complete.
- Defines job dependencies.

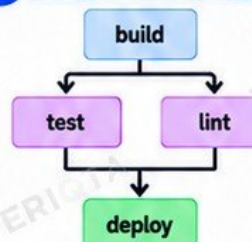
```
jobs:
  build:
    runs-on: ubuntu-latest
  deploy:
    needs: build
    runs-on: ubuntu-latest
```

5 EXIT CODES



- Every step returns an exit code.
- 0 = success
- Non-zero = failure
- If a step fails, the job stops and remaining steps are skipped (unless **continue-on-error** is set).

6 JOB DEPENDENCIES



- Use **needs** to create a clear execution order.
- A job can depend on multiple jobs.
- Downstream jobs run only if all required jobs succeed.

7 FAILED STEPS



- If a step fails, the job stops by default.
- You can allow the workflow to continue using **continue-on-error**.
- Check the logs to see the exact error.
- Fix the issue and re-run the workflow.

8 IF CONDITIONS



- Use **if** to run steps or jobs based on conditions.
- Common use cases:
 - Run only on main branch
 - Run only on pull requests
 - Run on success or failure

```
steps:
  - name: Deploy only on main
    if: github.ref == 'refs/heads/main'
    run: echo "Deploying..."
```

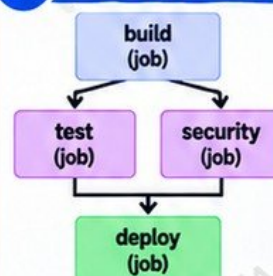
9 CONTINUE-ON-ERROR



- Allows a step to fail without stopping the job.
- Useful for non-critical steps (e.g. optional tests or reporting).

```
steps:
  - name: Run optional test
    continue-on-error: true
    run: npm test
```

10 WORKFLOW DEPENDENCY DIAGRAM



1. build runs first
2. test and security run in parallel (both need build)
3. deploy runs last (needs both)

11 KEY TAKEAWAYS



- Jobs run in parallel by default.
- Steps run sequentially.
- Use **needs** to control job order.
- Check exit codes and logs when something fails.

- Use **if** conditions to run only what is needed.
- Use **continue-on-error** for non-critical steps.
- Plan dependencies to make your workflow fast and reliable.
- A clear job structure makes troubleshooting easier.



JUNIOR ENGINEER TIP

A successful pipeline is not just about running many steps. It is about running the right steps, in the right order, and handling failures properly.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CODE • AUTOMATE • DEPLOY • GROW

Better
Engineers
Brighter
Futures

RUNNERS AND EXECUTION ENVIRONMENTS

The machines that run your workflows

1 WHAT A RUNNER ACTUALLY DOES



- A runner is a machine that executes your workflow jobs.
- It checks out your code, runs your steps, and sends the results back to GitHub.
- Each job runs on a fresh runner unless you use a self-hosted runner.

2 UBUNTU, WINDOWS, AND macOS RUNNERS



- Most commonly used
- Pre-installed with many tools
- Great for Linux applications



- Useful for .NET and Windows apps
- Runs on Windows Server
- Good for Windows-specific testing



- Used for iOS/macOS builds and testing
- Comes with Xcode
- Longer queue times (especially on free plans)

3 GITHUB-HOSTED RUNNERS



- Provided and maintained by GitHub.
- Available on Ubuntu, Windows, and macOS.
- Pre-installed with many popular tools (e.g. Node.js, Python, Docker, Git).
- Clean, fresh environment for each run (ephemeral).
- No infrastructure to manage.
- Ideal for most use cases.

4 SELF-HOSTED RUNNERS



- Hosted and managed by you.
- Can run on your own servers, VMs, or cloud.
- Useful for custom environments, private networks, or special hardware.
- You control the installed software.
- Requires maintenance, updates, and monitoring.
- Can be persistent (same machine reused).
- Use with caution, especially for public repositories.

5 RUNNER LABELS



- Runners have labels that you use in runs-on to select a runner.
- Examples: ubuntu-latest, windows-latest, macos-latest.
- Custom labels can be added to self-hosted runners.
- You can use multiple labels for more specific targeting.

```
jobs:
  build:
    runs-on: [self-hosted, linux, x64]
```

6 WORKSPACE LIFECYCLE



- A fresh workspace is created for each job on GitHub-hosted runners.
- Your repository is checked out into this workspace.
- The workspace is cleaned up after the job completes (ephemeral).
- On self-hosted runners, the workspace persists unless you manually clean it.

7 INSTALLED SOFTWARE



- GitHub-hosted runners come with many tools pre-installed (e.g. Git, Node.js, Python, Docker, Java, etc.).
- You can also install additional tools during your workflow using shell commands.
- Check the official documentation for the full list of installed software.

```
# Example: install a tool
- name: Install Terraform
  run: sudo apt-get install terraform
```

8 EPHEMERAL vs PERSISTENT RUNNERS

Feature	Ephemeral (GitHub-hosted)	Persistent (Self-hosted)
Lifetime	Created for each job	Stays running
Environment	Clean and fresh	Reused
Maintenance	Handled by GitHub	Handled by you
Data persistence	No (deleted after job)	Yes (remains)
Best for	Most use cases	Custom or special needs

9 SECURITY RISKS OF SELF-HOSTED RUNNERS



- Can be compromised if not properly secured.
- May expose your network or internal systems.
- Requires regular updates and patching.
- Risk of malicious pull requests running on your runner (if used with public repositories).
- Secrets and credentials must be carefully managed.
- Use network isolation, firewall rules, and least privilege.
- Monitor runner usage and access.



JUNIOR ENGINEER TIP

A runner is just a machine. Understanding its environment, tools, and limitations helps you write better workflows and avoid common issues.

Better
Engineers
Brighter
Futures



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Variables, Contexts, Expressions, and Outputs

Use data, share information, and make your workflows dynamic.

1 env (Environment Variables)

- env lets you define variables that can be used in your workflow.
- You can set env at workflow, job, or step level.
- Use it for non-sensitive configuration values.

```
env:
  APP_NAME: myapp
  ENVIRONMENT: production
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "App: $APP_NAME"
```

2 \${ } Expressions

- `${ }` is used to evaluate expressions and access context variables.
- You can use it in if conditions, inputs, env, and more.
- It allows you to use data from GitHub, runner, job, steps, etc.

```
steps:
  - name: Show branch
    run: echo "Branch: ${ github.ref }"

  - name: Use expression
    if: ${ github.event_name == 'push' }
    run: echo "This is a push event"
```

3 github Context

- Contains information about the workflow run, repository, event, actor, etc.
- Useful for conditionals and dynamic values.



```
steps:
  - run: |
      echo "Repository: ${ github.repository }"
      echo "Event: ${ github.event_name }"
      echo "Actor: ${ github.actor }"
      echo "SHA: ${ github.sha }"
```

4 runner Context

- Contains information about the runner that is executing the job.
- Useful for OS, architecture, runner name, etc.



```
steps:
  - run: |
      echo "OS: ${ runner.os }"
      echo "Runner: ${ runner.name }"
      echo "Architecture: ${ runner.arch }"
```

5 job Context

- Contains information about the current job.
- Useful for job status and outputs.



```
steps:
  - run: |
      echo "Job status: ${ job.status }"
      echo "Job ID: ${ job.job_id }"
```

6 steps Context

- Contains information about previous steps in the same job.
- Useful for accessing step outputs.



```
steps:
  - id: set-value
    run: echo "name=veriqta" >> $GITHUB_OUTPUT

  - run: echo "Name: ${ steps.set-value.outputs.name }"
```

7 needs Context

- Contains information about dependent jobs.
- Used to access outputs from other jobs.



```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      - version: ${ steps.get-version.outputs.version }
    steps:
      - id: get-version
        run: echo "version=1.0.0" >> $GITHUB_OUTPUT
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Version: ${ needs.build.outputs.version }"
```

8 Step Outputs

- Steps can set outputs using `$GITHUB_OUTPUT`.
- You must give the step an id to reference its outputs.



```
steps:
  - id: get-sha
    run: echo "sha=$GITHUB_SHA" >> $GITHUB_OUTPUT

  - run: echo "SHA: ${ steps.get-sha.outputs.shaa }"
```

9 Job Outputs

- Jobs can expose outputs that can be used by other jobs.
- Define outputs at the job level.



```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      - image: ${ steps.build.outputs.image }
    steps:
      - id: build
        run: echo "image=myapp:1.0.0" >> $GITHUB_OUTPUT
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Image: ${ needs.build.outputs.image }"
```

10 Passing Information Between Jobs

- Use job outputs and the needs context.
- Lets you pass values from one job to another.
- Common use cases: image tags, deployment URLs, build artifacts.

11 Common Expression Mistakes

- ✗ Forgetting `${ }` when using expressions
- ✗ Using quotes incorrectly
- ✗ Accessing the wrong context name
- ✗ Trying to use step outputs without an id
- ✗ Expecting environment variables to be available across jobs
- ✗ Using secrets or sensitive data in expressions (not allowed)



JUNIOR ENGINEER TIP

Automation does not remove the need to understand the deployment process. You still need to know how your application works, what is being deployed, and how the data flows. Variables, contexts, expressions, and outputs make your workflows flexible and powerful.

Same Skills
A Brighter
Future



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Secrets and Secure Credential Handling

Keep your credentials safe. Protect your systems. Build with confidence.

1 Repository Secrets

- Stored in a single repository.
- Available to all workflows in that repository.
- Used for repository-specific credentials (e.g. API keys, tokens).



```
# In your workflow
steps:
  - name: Use a repository secret
    run: echo "Token: ${ secrets.MY_TOKEN }"
```

2 Environment Secrets

- Stored in a specific environment (e.g. dev, staging, production).
- Only available to workflows that use that environment.
- Can be combined with protection rules and required approvals.



```
jobs:
  deploy:
    environment: production
    steps:
      - run: echo "DB: ${ secrets.DB_PASSWORD }"
```

3 Organization Secrets

- Stored at the organization level.
- Available to multiple repositories within the organization.
- Useful for shared credentials (e.g. cloud accounts, third-party tools).



```
steps:
  - name: Use an org secret
    run: echo "Key: ${ secrets.ORG_API_KEY }"
```

4 \${{ secrets.NAME }}

- Use this syntax to access secrets in your workflows.
- Secrets are encrypted and injected at runtime.
- They are not visible in your repository or workflow logs (when properly handled).

```
steps:
  - name: Use a secret
    run: echo "Connecting..."
    env:
      API_KEY: ${ secrets.API_KEY }
```



5 Why Credentials Must Never Be Hardcoded

- Hardcoded credentials can be easily seen, copied, and abused.
- They may end up in your Git history and stay there forever.
- Exposed credentials can lead to data breaches, financial loss, and infrastructure compromise.



NEVER HARDCODE CREDENTIALS IN YOUR CODE.

6 Secret Masking Limitations

- GitHub masks exact secret values in logs, but it is not foolproof.
- If you transform or partially print a secret, it may not be masked.
- Secrets in debug logs, error messages, or encoded formats might still be exposed.
- Always avoid printing secrets, even partially.



Do not rely only on masking. Handle secrets carefully.

7 Least Privilege

- Give only the minimum access required.
- Use scoped tokens and limited permissions.
- Avoid using personal access tokens (PATs) when not necessary.
- Regularly review and rotate credentials.



Minimum access. Maximum security.

8 Protecting Tokens

- Use GitHub's built-in GITHUB_TOKEN when possible.
- Set explicit permissions for jobs.
- Use short-lived credentials (e.g. OIDC) for cloud access.
- Rotate tokens regularly.
- Do not share tokens or store them outside GitHub.



Protect your tokens like you protect your infrastructure.

9 Avoiding Secrets in Logs

- Do not echo secrets.
- Avoid using set -x in scripts.
- Be careful with error messages that may include sensitive data.
- Sanitize logs and remove sensitive information.
- Review workflow logs regularly.



Logs are not private. Keep sensitive data out of them.

10 Security Reminder



- Treat all secrets as sensitive information.
- Use least privilege, environment protection, and secure practices.
- A single exposed credential can compromise your code, data, and infrastructure.
- Build securely, always.

SECURITY IS A HABIT, NOT A FEATURE.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Permissions and GITHUB_TOKEN Security

Control access. Reduce risk. Build secure workflows.

1 What GITHUB_TOKEN Is

- GITHUB_TOKEN is an automatically generated token provided by GitHub for each workflow run.
- It allows your workflow to authenticate with the GitHub API.
- Its permissions are limited and scoped to the repository by default.
- You do not need to create or store this token.



2 Workflow Permissions

- You can control what the GITHUB_TOKEN can do using the permissions key.
- Set permissions at the workflow or job level.
- Only grant the minimum permissions required.



```
permissions:
  contents: read
  packages: write
```

3 contents: read

- Allows read access to repository contents (code, files, commits).
- Required for common tasks like checking out code.
- Use read instead of write unless you need to push changes.



```
permissions:
  contents: read
```

4 packages: write

- Allows pushing packages to GitHub Packages (e.g. container images).
- Required when your workflow builds and publishes packages.
- Do not enable unless needed.



```
permissions:
  contents: read
  packages: write
```

5 Pull Request Permissions

- Workflows triggered by pull requests from forks have restricted permissions.
- GITHUB_TOKEN has read-only access by default in forked pull requests.
- This helps prevent malicious code from gaining write access to your repository.



6 Job-Level Permissions

- You can set different permissions for each job.
- Useful when some jobs need more access than others.



```
jobs:
  build:
    permissions:
      contents: read
  publish:
    permissions:
      contents: read
      packages: write
```

7 Least Privilege

- Grant only the permissions required for the job to succeed.
- Avoid broad permissions.
- Reduces the impact if a workflow is compromised.
- Regularly review and update permissions.



Minimum permissions.
Maximum security.

8 Why write-all Can Be Dangerous

- write-all gives the workflow full write access to many resources (code, packages, issues, etc.).
- If a workflow is compromised, attackers can push malicious code, delete resources, or modify settings.
- Only use broad permissions in trusted, controlled workflows.



Do not use write-all
unless absolutely necessary.

9 Untrusted Pull Requests

- Pull requests from forks can contain malicious code.
- They run in your workflow environment.
- Never give write permissions to untrusted pull requests.
- Use conditions or separate workflows for sensitive tasks (e.g. deployments).



Treat all external contributions
as untrusted.

10 Production Permission Strategy



- Start with read-only permissions.
- Grant additional permissions only when required (e.g. packages: write for publishing).
- Use environment protection rules for deployments.
- Review permissions regularly.
- Keep sensitive tasks (deployments, releases) in trusted workflows and branches.

Secure workflows
deliver reliable
software.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Build a Real Continuous Integration Pipeline

Automate the steps. Get fast feedback. Build with confidence.

1 Checkout Source Code



- Checks out your repository so the workflow can access your code.
- Usually uses the official action.

```
- name: Checkout code
  uses: actions/checkout@v4
```

2 Set Up Runtime

- Sets up the required runtime environment (e.g. Node.js, Python, Java, etc.).
- Uses official setup actions.



```
- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
```

3 Install Dependencies

- Installs all required project dependencies.
- Uses package managers like npm, pip, Maven, etc.



```
- name: Install dependencies
  run: npm ci
```

4 Lint

- Runs a linter to check code quality and style.
- Helps catch issues early before running tests.



```
- name: Run linter
  run: npm run lint
```

5 Unit Test

- Runs automated tests to verify the application works as expected.
- Builds confidence and prevents regressions.



```
- name: Run tests
  run: npm test
```

6 Build

- Builds your application (e.g. compiles code, creates a distributable).
- The exact command depends on your project.



```
- name: Build application
  run: npm run build
```

7 Package

- Creates a package or artifact (e.g. JAR, ZIP, Docker image, etc.).
- Prepares the output for deployment or distribution.



```
- name: Package application
  run: npm run package
```

8 Upload Artifacts

- Uploads build artifacts so they can be downloaded and used in later jobs or for deployment.
- Uses the upload-artifact action.



```
- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: build-output
    path: dist/
```

9 Fail Fast

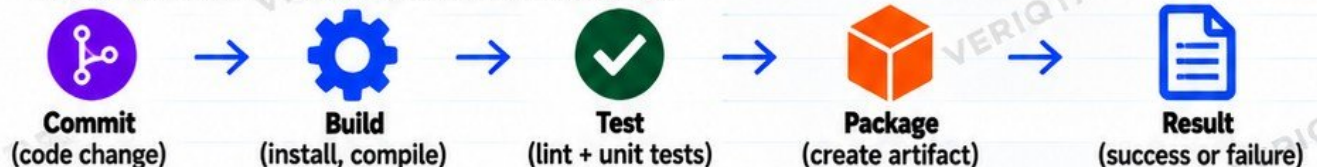
- The pipeline stops immediately if a step fails.
- Prevents wasting time and resources.
- Helps you fix issues early.



```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: npm test
        continue-on-error: false
```

10 CI Flow

A typical continuous integration pipeline follows these steps:



JUNIOR ENGINEER TIP

A good CI pipeline gives you fast feedback, keeps your code healthy, and makes deployments safer and easier.

“FAST FEEDBACK
BUILDS
BETTER SOFTWARE.”



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD • LEARN • AUTOMATE • GROW

Same Skills
A Brighter You

ARTIFACTS, CACHING, AND PIPELINE EFFICIENCY

Store, reuse, and manage data in your workflows for faster, more reliable builds.

1 WORKFLOW ARTIFACTS



- Artifacts are files generated during a workflow run.
- They are stored by GitHub and can be downloaded later.
- Useful for build outputs, logs, reports, and test results.
- Artifacts are tied to a specific workflow run.

2 UPLOADING ARTIFACTS

Use the `upload-artifact` action to store files.

```
- name: Upload build artifact
  uses: actions/upload-artifact@v4
  with:
    name: my-app
    path: |
      dist/
      build/
```

3 DOWNLOADING ARTIFACTS

Use the `download-artifact` action to retrieve files in a later job.

```
- name: Download artifact
  uses: actions/download-artifact@v4
  with:
    name: my-app
    path: ./artifacts
```

4 DEPENDENCY CACHING



- Caching stores dependencies to avoid downloading them every time.
- Speeds up builds significantly.
- Common examples: npm, pip, maven, gradle, docker layers.

5 CACHE KEYS



- A cache key uniquely identifies the cache.
- Use a combination of OS, package file, and version.
- If the key changes, a new cache is created.

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{
      hashFiles('package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-
```

6 CACHE HITS AND MISSES



Cache Hit

Dependencies are restored from the cache. Build is faster.



Cache Miss

No matching cache found. Dependencies are downloaded and a new cache is created.

7 ARTIFACT vs CACHE

ARTIFACT

- Stores build outputs (files, logs, reports)
- Used between jobs or for download
- Immutable (read-only)
- Kept for a limited time (retention period)

CACHE

- Stores dependencies
- Used to speed up future runs
- Mutable (can be overwritten)
- Automatically managed by key

8 RETENTION



- Artifacts are stored for a limited time (default 90 days).
- You can set a custom retention period (1-90 days).
- Caches also have a retention period and may be evicted automatically when not used.

```
- uses: actions/upload-artifact@v4
  with:
    name: my-app
    path: dist/
    retention-days: 14
```

9 WHY INCORRECT CACHING CAUSES CONFUSING BUILDS



- Using the wrong cache key can restore the wrong dependencies.
- Stale or incompatible dependencies can cause unexpected build failures.
- Not including important files in the key (e.g. lock files) leads to cache collisions.
- Caches can be corrupted if not managed properly.
- Always use precise and meaningful cache keys.

10 REPRODUCIBLE PIPELINES



- Use lock files (package-lock.json, requirements.txt, etc.).
- Combine caching with versioned dependencies.
- Ensure builds produce the same result every time.
- Avoid relying on external, changing dependencies.
- Reproducible pipelines make your deployments more reliable and predictable.



JUNIOR ENGINEER TIP

Cache what you can, artifact what you need. Use correct keys and version control to avoid confusing builds.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD • LEARN • AUTOMATE • GROW

Same
Skills
Bigger
Opportunities

MATRIX BUILDS AND MULTI-ENVIRONMENT TESTING

Test your application across multiple versions, operating systems, and environments in parallel.

1 STRATEGY.MATRIX

- **strategy.matrix** lets you run the same job with different combinations of variables.
- It helps you test across multiple versions, operating systems, or configurations at the same time.
- Each combination runs as a separate job.

2 TESTING MULTIPLE VERSIONS

Example: Test a Node.js application on multiple versions.

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${matrix.node-version}
      - run: npm ci
      - run: npm test
```

3 MULTIPLE OPERATING SYSTEMS

Example: Test on Ubuntu, Windows and macOS.

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-latest]
    node-version: [18, 20]
runs-on: ${matrix.os}
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: ${matrix.node-version}
  - run: npm test
```

4 MATRIX COMBINATIONS

- GitHub Actions creates a job for every combination of the matrix variables.
- If you have 3 Node.js versions and 3 operating systems, you get $3 \times 3 = 9$ jobs.
- All jobs run in parallel by default.

Example:

```
node-version: [18, 20, 22]
os: [ubuntu-latest, windows-latest, macos-latest]
Total jobs: 9
```

5 INCLUDE

- Use **include** to add extra combinations that are not part of the main matrix.

```
strategy:
  matrix:
    node-version: [18, 20]
    os: [ubuntu-latest]
  include:
    - node-version: 22
      os: ubuntu-latest
      experimental: true
```

6 EXCLUDE

- Use **exclude** to skip specific combinations that you don't want to run.

```
strategy:
  matrix:
    node-version: [18, 20, 22]
    os: [ubuntu-latest, windows-latest]
  exclude:
    - node-version: 18
      os: windows-latest
    - node-version: 22
      os: ubuntu-latest
```

7 FAIL-FAST

- By default, **fail-fast** is true. If one job fails, the remaining jobs are cancelled.
- Set **fail-fast: false** to let all jobs run, even if some fail.

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
    node-version: [18, 20, 22]
```

8 PARALLEL EXECUTION

- Matrix jobs run in parallel, which saves time.
- Each job gets its own runner and environment.
- Example: 9 jobs can run at the same time instead of one after another.



9 WHEN MATRIX BUILDS BECOME WASTEFUL

- ✗ Testing too many combinations can be slow and expensive.
- ✗ Some combinations may not add real value (e.g. unsupported versions).
- ✗ Running the same tests on many OS versions when your application is platform-specific.
- ✗ Using large test suites for every combination.
- ✗ It's important to balance coverage with build time and resource usage.



Be intentional. Test what matters.

10 PRACTICAL APPLICATION TESTING EXAMPLE

Example: Test a Python application on multiple Python versions and OS.

```
jobs:
  test:
    runs-on: ${matrix.os}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
        python-version: [3.9, 3.10, 3.11]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${matrix.python-version}
      - run: pip install -r requirements.txt
      - run: pytest -v
```



JUNIOR ENGINEER TIP

Matrix builds are powerful, but don't test everything just because you can. Focus on the versions and environments that your application actually supports and that give you real confidence.



A good pipeline is fast, reliable, and efficient.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Same
Knowledge
Higher
Opportunities

DOCKER BUILDS AND CONTAINER REGISTRY AUTOMATION

Build, tag, and **push** container images automatically in your workflows.

1 BUILD CONTAINER IMAGE



- Build a container image from your application using a Dockerfile.
- This packages your application and its dependencies into a portable image.

```
- name: Build Docker image
  run: |
    docker build -t my-app .
```

2 AUTHENTICATE TO REGISTRY



- Authenticate securely using a token or GitHub-provided credentials.
- For GitHub Container Registry (GHCR), use `GITHUB_TOKEN`.

```
- name: Login to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}
```

3 IMAGE TAGGING



- Tag your image so it can be identified and versioned.
- Use meaningful tags instead of just latest.

```
- name: Tag image
  run: |
    docker tag my-app:latest \
      ghcr.io/${{ github.repository }}:
        ${{ github.sha }}
```

4 COMMIT SHA TAGS



- Use the commit SHA to create a unique and immutable tag for each build.
- This ensures you can always trace an image back to the exact code.

```
docker tag my-app:latest \
  ghcr.io/${{ github.repository }}:
    ${{ github.sha }}
```

5 SEMANTIC VERSION TAGS



- Use semantic versioning (e.g. v1.2.3) for release images.
- Helps with stable releases and rollbacks.

```
docker tag my-app:latest \
  ghcr.io/${{ github.repository }}:
    v1.2.3
```

6 PUSH IMAGE



- Push your tagged image to the registry.
- You can push multiple tags (e.g. commit SHA and version).

```
- name: Push image
  run: |
    docker push ghcr.io/${{ github.repository }}:${{ github.sha }}
    docker push ghcr.io/${{ github.repository }}:v1.2.3
```

7 GITHUB CONTAINER REGISTRY



- GitHub Container Registry (GHCR) is a container registry built into GitHub.
- Use the registry URL: `ghcr.io`.
- Images can be private or public.
- Access is controlled using GitHub permissions and tokens.

```
ghcr.io/owner/repository:tag
```

8 AVOIDING LATEST AS YOUR ONLY IDENTIFIER



- Do not rely only on latest for production.
- The latest tag can change at any time, making deployments unpredictable.
- Use specific tags (commit SHA, version, or both).
- Keep latest only as an additional tag if needed (e.g. for non-production environments).

9 CI/CD PIPELINE EXAMPLE



Git
(Commit)



Test
(Build & Test)



Docker Build
(Build & Tag)



Container Registry
(Push Image)

10 JUNIOR ENGINEER TIP



Use meaningful tags (commit SHA and semantic versions) and avoid relying only on latest. It makes your deployments reliable, traceable, and easier to roll back.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

REUSABLE WORKFLOWS AND CUSTOM ACTIONS

Write once. Reuse. Automate at scale.

1 WHY DUPLICATED YAML BECOMES DIFFICULT TO MAINTAIN



- Copying the same steps across multiple repositories leads to:
 - Inconsistent configurations
 - Harder updates
 - More errors
 - Time wasted
 - Difficult to enforce best practices

DRY (Don't Repeat Yourself) makes your automation cleaner, safer, and easier to maintain.

2 REUSABLE WORKFLOWS



- Reusable workflows let you define a workflow once and use it in multiple repositories.
- Stored in a repository (often in a central repo).
- Called using the `workflow_call` event.
- Supports inputs and secrets.

Perfect for standard CI/CD pipelines, security scans, or deployment workflows.

3 workflow_call



- The `workflow_call` event allows a workflow to be called by another workflow.
- You can define inputs and secrets for flexibility.

```
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
    secrets:
      token:
        required: true
```

4 INPUTS



- Inputs allow you to pass values (e.g. environment, version, region) into a reusable workflow.
- Define the type (string, boolean, number) and whether it is required.

```
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
```

5 SECRETS



- Secrets allow you to securely pass sensitive information to reusable workflows.
- Secrets are defined in the caller and accessed in the called workflow using `{{ secrets.NAME }}`.

```
on:
  workflow_call:
    secrets:
      token:
        required: true
```

6 CALLING REUSABLE WORKFLOWS



- Use the `uses` keyword to call a reusable workflow from another workflow.
- Pass inputs and secrets as needed.

```
jobs:
  build:
    uses: org/repo/.github/workflows/ci.yml@v1
    with:
      environment: production
    secrets:
      token: {{ secrets.GITHUB_TOKEN }}
```

7 COMPOSITE ACTIONS



- Composite actions let you combine multiple steps into a single action.
- Stored in your repository under `.github/actions/`.
- Useful for reusable logic like setup, configuration, or common tasks.

```
runs:
  using: "composite"
  steps:
    - run: echo "Setup complete"
```

8 MARKETPLACE ACTIONS



- You can use actions from the GitHub Marketplace.
- Examples: `actions/checkout`, `actions/setup-node`, `docker/login-action`.
- These are maintained by the community or GitHub.
- Always review the source and trust level before using.

9 PINNING THIRD-PARTY ACTIONS



- Always pin actions to a specific version to avoid unexpected changes.
- Use a full commit SHA for maximum security.

```
# Less secure
uses: actions/checkout@v3

# More secure
uses: actions/checkout@a1b2c3d4e5f6...
```

10 VERSIONING INTERNAL AUTOMATION



- Use semantic versioning (e.g. v1.0.0) for your internal actions and reusable workflows.
- Tag releases in Git.
- Update versions in dependent repositories deliberately.
- Maintain a changelog to track changes.

Treat your automation like production code.



JUNIOR ENGINEER TIP

Reusable workflows and custom actions help you work smarter, not harder. Standardize, version, and keep your automation clean, secure, and easy to maintain.

**AUTOMATE ONCE.
USE EVERYWHERE.
THAT'S LEVERAGE.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD • LEARN • AUTOMATE • GROW

Same
Knowledge
Higher
Opportunities

ENVIRONMENTS, APPROVALS, AND DEPLOYMENT PROTECTION

Control how and where your application is deployed. Add the right checks before production.

1 GITHUB ENVIRONMENTS



- GitHub environments let you define deployment targets with protection rules, secrets, and approvals.
- Common environments:

Development	For feature development and testing.
Staging	For final testing before production.
Production	Live environment for real users.

2 ENVIRONMENT-SPECIFIC CONFIGURATION



- Use different configuration per environment.
- Store environment-specific values as variables or secrets.
- Keep configs separate to avoid accidental production changes.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: echo "Deploying to ${github.environment}"
```

3 REQUIRED REVIEWERS



- Require one or more reviewers before a deployment can proceed.
- Helps prevent accidental or unauthorized deployments.

Environment protection rules

- ☒ Required reviewers
Specify who can approve deployments to this environment.

Q Add reviewers

- platform-team
- sre-team
- your-username

4 DEPLOYMENT APPROVALS



- Deployments can be paused until a reviewer approves.
- You can set a timeout for pending approvals.
- Approvals add a manual check before changes reach sensitive environments.

5 BRANCH RESTRICTIONS



- Restrict which branches can deploy to an environment.
- Common setup: only the main branch can deploy to production.

Environment protection rules

- ☒ Deployment branches
Limit which branches can deploy to this environment.

main

e.g. main, release/*

6 ENVIRONMENT SECRETS



- Store sensitive data per environment.
- Each environment can have its own secrets.
- Keeps production credentials separate from development.

```
jobs:
  deploy:
    environment: production
    steps:
      - run: echo "Using secret"
    env:
      API_KEY: ${ secrets.PROD_API_KEY }
```

7 PRODUCTION GATES



- Combine multiple protection rules for maximum safety.
- Examples:
 - Required reviewers
 - Branch restrictions
 - Environment secrets
 - Wait timer (cool-down period)
- These gates reduce risk and ensure controlled releases.

8 WHY PASSING CI IS NOT ENOUGH



- Passing CI only means the code built and tested successfully.
- It does NOT mean:
 - It is safe to deploy
 - It is the right version
 - It is approved
 - It has been reviewed for risks
 - It won't break production

Always add protection rules for production.
Automation should be fast, not careless.

9 EXAMPLE DEPLOYMENT JOB



```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - run: echo "Deploying to production..."
      - run: ./deploy.sh
```



JUNIOR ENGINEER TIP

Use environments and approvals to protect your users, your data, and your reputation. Control is a feature, not a delay.



**DEPLOY SLOWLY
TO MOVE FASTER
IN THE LONG RUN.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DEPLOYING TO KUBERNETES

From built image to a running application in your Kubernetes cluster.

1 CI vs CD RESPONSIBILITIES

CI (Continuous Integration)

- ✓ Build the application
- ✓ Run tests
- ✓ Check code quality
- ✓ Build container image
- ✓ Push image to registry
- ✓ Produce artifacts

CI ensures the application is ready to deploy.

CD (Continuous Deployment)

- ✓ Deploy to environments
- ✓ Manage configuration
- ✓ Use Helm or Kubernetes manifests
- ✓ Verify deployment
- ✓ Handle rollbacks
- ✓ Monitor the application

CD takes the built application and releases it to Kubernetes.

2 DEPLOYMENT FLOW



3 BUILD APPLICATION IMAGE



- Use a Dockerfile to build your image.
- Tag the image with a version (e.g., commit SHA).
- Build the image in your CI pipeline.

```
# Build the image
docker build -t myapp:1.0.0 .
```

4 PUSH TO REGISTRY



- Push the image to a container registry (e.g., Docker Hub, ECR, GCR, or ACR).
- Use a unique tag (avoid only latest).

```
# Tag and push
docker tag myapp:1.0.0 <registry>/myapp:1.0.0
docker push <registry>/myapp:1.0.0
```

5 AUTHENTICATE SECURELY



- Use secrets or OIDC for secure authentication.
- Never hardcode credentials in your workflow.
- Use least privilege permissions.

```
# Example: AWS ECR login
aws ecr get-login-password |
docker login --username AWS
--password-stdin <account>.dkr.ecr.
<region>.amazonaws.com
```

6 HELM DEPLOYMENT



- Use Helm to deploy your application.
- Install or upgrade the release.
- Use values files for environment configuration.

```
# Install or upgrade
helm upgrade --install myapp ./myapp
--namespace myapp --create-namespace
-f values-prod.yaml
```

7 KUBERNETES NAMESPACE



- Use a dedicated namespace for your application.
- Create the namespace if it does not exist.
- Isolate environments (dev, staging, prod).

```
# Create namespace
kubectl create namespace myapp
```

8 DEPLOYMENT VERIFICATION



- Check the release status.
- Verify pods, services, and ingress.
- Make sure the application is healthy.

```
# Helm status
helm status myapp -n myapp
# Check resources
kubectl get pods,svc,ingress -n myapp
```

9 USEFUL COMMANDS



- Deploy with Helm:


```
helm upgrade --install myapp ./myapp
```
- Check rollout status:


```
kubectl rollout status deployment/myapp -n myapp
```
- View pods:


```
kubectl get pods -n myapp
```
- View logs:


```
kubectl logs -f deployment/myapp -n myapp
```

10 FAILURE HANDLING



- Check pod status and logs.
- Look for image pull errors.
- Verify resource limits and configuration.
- Check Helm release history.
- Identify misconfigurations in values files.
- Fix the issue and redeploy.

11 ROLLBACK PLANNING



- Keep previous working versions.
- Use Helm rollback if needed.
- Verify the application after rollback.
- Have a clear rollback strategy for production.

```
# Roll back to previous release
helm rollback myapp -n myapp
```



JUNIOR ENGINEER TIP

A successful deployment is not just about running a command. Always verify the application, monitor it, and have a rollback plan in place. Automate the process, but understand what is happening behind the scenes.

DEPLOY
VERIFY
MONITOR
BE READY



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CLOUD AUTHENTICATION WITH OIDC

Secure, short-lived credentials for modern DevOps pipelines.

1 WHAT OIDC MEANS



- OpenID Connect (OIDC) is an identity layer built on top of the OAuth 2.0 standard.
- It allows a trusted identity provider (GitHub) to issue a digitally signed token that proves who you are.
- In CI/CD, GitHub issues an identity token for your workflow instead of using long-lived cloud credentials.

2 THE PROBLEM WITH LONG-LIVED CLOUD CREDENTIALS



- Static access keys are hard to manage and often stored as secrets.
- They can be leaked, misused, or accidentally exposed in logs or code.
- They do not expire automatically.
- Rotating keys is manual and often forgotten.
- Long-lived credentials increase the risk of unauthorized access.

3 GITHUB IDENTITY TOKEN



- When a workflow runs, GitHub can issue a short-lived OIDC token (JWT).
- The token contains claims about your repository, workflow, branch, and other context.
- This token is used to authenticate to your cloud provider.
- No static keys are stored in your repository.

4 CLOUD TRUST RELATIONSHIP



- You configure a trust relationship in your cloud provider (AWS, Azure, or GCP).
- The cloud provider trusts GitHub as an identity provider based on specific conditions (e.g., repository, branch, environment).
- If the conditions match, the cloud provider accepts the OIDC token and issues temporary credentials.

5 SHORT-LIVED CREDENTIALS



- The cloud provider exchanges the OIDC token for short-lived credentials (e.g., temporary access keys or tokens).
- These credentials are valid for a limited time (usually minutes to hours).
- They expire automatically and cannot be reused.
- No manual rotation is required.

6 AWS, AZURE, AND GOOGLE CLOUD PATTERNS



- Use IAM OIDC provider.
- Create an IAM role with trust policy for GitHub.
- Use role assumption to get temporary credentials.



- Use Workload Identity Federation.
- Create a federated credential.
- Assign an Azure role (e.g., Contributor).
- Use the OIDC token to get an access token.



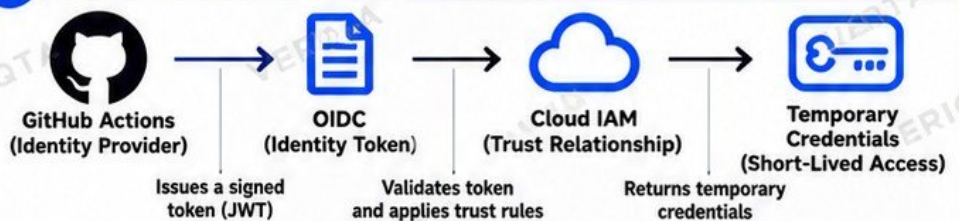
- Use Workload Identity Federation.
- Create a workload identity pool and provider.
- Grant IAM roles (e.g., roles/container.developer).
- Exchange OIDC token for temporary credentials.

7 LEAST PRIVILEGE



- Grant only the permissions required for the specific task.
- Use separate roles for different environments (dev, staging, production).
- Limit access to specific repositories, branches, or environments.
- Regularly review and audit permissions.

8 ARCHITECTURE DIAGRAM



No long-lived credentials. No secrets in your repository. Just secure, temporary access.

9 WHY OIDC IS PREFERRED FOR MODERN PRODUCTION PIPELINES



- More secure: no long-lived credentials to leak.
- Simpler management: no manual key rotation.
- Granular access control using repository, branch, and environment conditions.
- Works across all major cloud providers.
- Integrates seamlessly with modern CI/CD pipelines.
- Reduces the risk of credential exposure.
- Follows zero trust and least privilege principles.



JUNIOR ENGINEER TIP

OIDC does not remove the need to understand your cloud provider's IAM system. You still need to know what permissions are required and how your application accesses cloud resources.

IDENTITY
INTEGRATION
FOR A MORE
SECURE TOMORROW



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN • BUILD • AUTOMATE • DEPLOY • GROW

Same
Knowledge
Bigger
Possibilities

TROUBLESHOOTING FAILED WORKFLOWS

Find the issue. Understand the cause. Fix it with confidence.

1 READING WORKFLOW LOGS



- Open the workflow run.
- Click on the failed job.
- Read the logs from top to bottom.
- Look for error messages in **red**.
- Check the full log, not just the last few lines.

```
# Run npm install
...
Error: Process exited
with code 1
```

2 FINDING THE FAILED JOB



- A workflow can have multiple jobs.
- The failed job will be marked in **red**.
- Click on the job to view its logs.
- Check job dependencies (using needs).

✗	build	12s
✓	test	18s
✗	deploy	5s

3 FINDING THE FAILED STEP



- Each job has multiple steps.
- The failed step is marked in **red**.
- Click on the step to see detailed logs.
- Look at the exact error message and the lines before it.

✓	Set up job
✓	Checkout code
✗	Run tests
⌚	Post job

4 EXIT CODES



- Exit codes indicate success or failure.
- 0 = success
- Non-zero = failure
- Common exit codes:

1	General error
126	Permission denied
127	Command not found
128	Git error
137	Out of memory
255	Unknown error

5 MISSING SECRETS



- Check that the secret name is correct.
- Ensure the secret exists in the right scope (repository, environment, or organization).
- Verify it is referenced correctly in the workflow.

```
Error: Secret MY_SECRET
is not set
```

6 PERMISSION DENIED



- Check if the GITHUB_TOKEN has the required permissions.
- Verify repository or organization settings.
- Ensure the workflow has the correct permissions in the YAML file.
- For cloud access, check IAM roles and policies.

```
Error: Permission denied
(fatal: could not read
Username for 'https://github.com')
```

7 DEPENDENCY FAILURES



- Check if dependencies are installed correctly.
- Verify version compatibility.
- Look for network issues when downloading packages.
- Check if a required service is available (e.g., database).

```
npm ERR! code E404
npm ERR! 404 Not Found -
package not found
```

8 YAML AND EXPRESSION ERRORS



- Check YAML syntax (indentation, colons, quotes).
- Verify expressions inside `${{ }}`.
- Ensure all variables and contexts are correct.
- Use the GitHub Actions workflow syntax reference for validation.

```
Error: The workflow is not valid.
(.github/workflows/ci.yml)
Line 12: Unexpected value 'steps'
```

9 RUNNER PROBLEMS



- Check if the runner is online and available.
- For self-hosted runners, verify the runner is registered and not offline.
- Look for disk space, memory, or network issues.
- Check runner labels (e.g., ubuntu-latest).

```
Error: Runner does not have
enough disk space
```

10 DOCKER BUILD FAILURES



- Check the Dockerfile syntax.
- Ensure the base image exists.
- Verify network access for package installation.
- Check if build arguments or secrets are provided.
- Look at the full build log for the exact error.

```
Error: failed to solve:
process "/bin/sh -c npm install"
exited with code 1
```

11 DEPLOYMENT FAILURES



- Check cluster connectivity and credentials.
- Verify namespace exists.
- Check Helm values and configurations.
- Look at pod status and events in Kubernetes.
- Review application logs for runtime errors.

```
Error: UPGRADE FAILED:
failed to create resource(s):
namespace "myapp" not found
```

12 RE-RUNNING JOBS



- You can re-run the failed job from the GitHub UI.
- Use "Re-run jobs" or "Re-run failed jobs".
- After fixing the issue, re-run to confirm the fix.
- Check if the problem is consistent or intermittent.

- Re-run all jobs
- Re-run failed jobs

13 DEBUGGING WITH EVIDENCE



- Read the logs carefully.
- Identify the exact error message.
- Check what changed recently (code, dependencies, secrets, configuration).

- Reproduce the issue locally when possible.
- Use small, targeted changes.
- Fix the root cause, not just the symptom.
- Document what you learn.

DON'T GUESS.
USE EVIDENCE.
THAT'S HOW
ENGINEERS SOLVE
PROBLEMS.



JUNIOR ENGINEER TIP

- Don't randomly edit YAML or rerun the workflow without understanding the error.
- Take time to read the logs, find the real cause, and fix it properly. This will make you a stronger engineer.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN • BUILD • AUTOMATE • GROW

Same Skills
Bigger
Opportunities

PRODUCTION PIPELINE SECURITY AND RELIABILITY

Build secure, reliable, and trustworthy pipelines for production.

1 LEAST-PRIVILEGE PERMISSIONS



- Grant only the permissions required for the job.
- Avoid write-all permissions.
- Use job-level permissions.
- Limit access to specific resources.

```
permissions:
  contents: read
  packages: write
  id-token: write
```

2 DEPENDENCY PINNING



- Use specific versions for actions, not floating tags.
- Avoid using @main or @latest.
- Helps prevent unexpected changes or malicious updates.
- Regularly review and update versions.

```
- uses: actions/checkout@v4
- uses: docker/setup-buildx-action@v3
```

3 SUPPLY-CHAIN RISKS



- Third-party actions can introduce risks.
- A compromised action can affect your pipeline and infrastructure.
- Use trusted sources.
- Review the action's repository, maintainers, and activity.

4 UNTRUSTED ACTIONS



- Only use actions from trusted authors.
- Check the number of stars, contributors, and recent updates.
- Avoid unverified or unknown actions.
- Consider using self-hosted or internal actions for sensitive tasks.

5 SECRET EXPOSURE



- Never hardcode secrets in your workflow.
- Use GitHub Secrets or OIDC.
- Be aware of secret masking limitations.
- Avoid printing sensitive data in logs.

```
env:
  DB_PASSWORD: ${ secrets.DB_PASSWORD }
```

6 PULL REQUEST SECURITY



- Treat pull requests from forks as untrusted.
- Use read-only permissions for PR workflows.
- Avoid exposing secrets to untrusted code.
- Use conditions to restrict sensitive jobs.

```
if: github.event.pull_request
  .head.repo.fork == false
```

7 PROTECTED ENVIRONMENTS



- Use environments for staging and production.
- Require reviewers for production deployments.
- Use environment-specific secrets.
- Protect against accidental deployments.

8 CONCURRENCY CONTROLS



- Prevent multiple workflows from running at the same time.
- Avoid overlapping deployments.
- Use the concurrency key.

```
concurrency:
  group: prod-deploy
  cancel-in-progress: true
```

9 TIMEOUTS



- Set timeouts for jobs and steps.
- Prevent stuck or hanging workflows.
- Save runner minutes and resources.

```
jobs:
  build:
    timeout-minutes: 30
```

10 DEPLOYMENT VERIFICATION



- Verify the deployment was successful.
- Check application health, services, and endpoints.
- Use automated checks (e.g., kubectl rollout status).
- Fail the pipeline if verification fails.

```
kubectl rollout status
deployment/myapp -n prod
```

11 AUDITABILITY



- Keep a record of who deployed what and when.
- Use GitHub's audit logs and environment history.
- Tag releases and use meaningful commit messages.
- Helps with compliance and incident investigations.

12 ROLLBACK STRATEGY



- Have a clear rollback plan before deploying.
- Automate rollbacks when possible.
- Test your rollback process regularly.
- Keep previous versions available.

```
# Example: Helm rollback
helm rollback myapp -n prod 1
```

13 PRODUCTION PIPELINE CHECKLIST



- | | |
|--|--|
| <ul style="list-style-type: none"> Use least-privilege permissions Pin action versions Review third-party actions Protect secrets Secure pull request workflows Use protected environments Implement concurrency controls | <ul style="list-style-type: none"> Set appropriate timeouts Verify deployments automatically Enable audit logs and tracking Have a rollback strategy Test failure scenarios Document the pipeline Regularly review and improve security |
|--|--|

SECURE
RELIABLE
AUTOMATED
READY FOR
PRODUCTION



JUNIOR ENGINEER TIP

- Security and reliability are not optional in production. Build pipelines like an engineer who will be on-call. Assume something will fail, and design for safety, visibility, and quick recovery.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN • BUILD • AUTOMATE • GROW

Same Skills
Bigger
Opportunities

FINAL PRODUCTION LAB

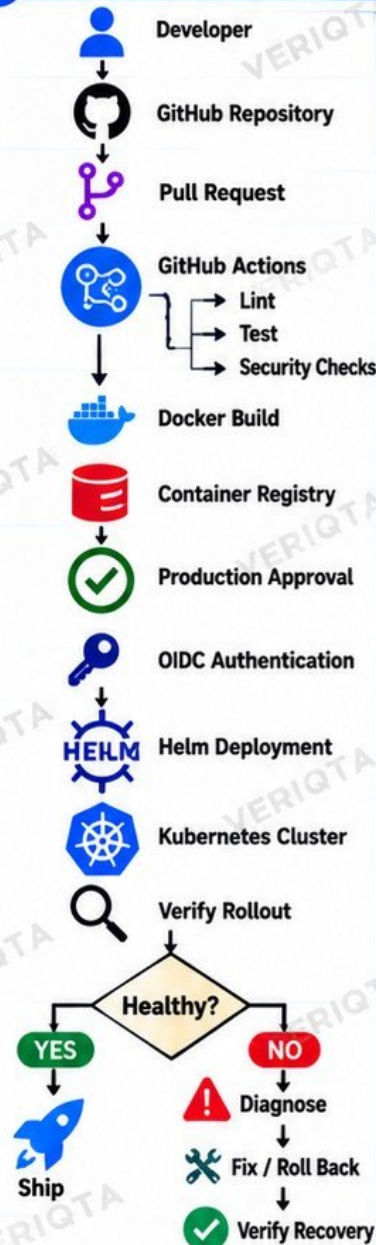
Build to Kubernetes – From Code to Production

A hands-on lab to put everything together and build a real, production-ready pipeline.

1 LAB STEPS

- 1  **Create application repository**
Initialize a new repository with your application code.
- 2  **Build a production workflow**
Create a complete CI/CD workflow file.
- 3  **Trigger on pull request**
Run the workflow when a pull request is opened.
- 4  **Run linting and tests**
Add steps for code quality and automated tests.
- 5  **Build application**
Compile or package your application.
- 6  **Build Docker image**
Create a container image for your application.
- 7  **Authenticate without hardcoded credentials**
Use OIDC for secure cloud authentication.
- 8  **Push image to registry**
Push the image to a container registry (e.g., ECR, ACR, GCR).
- 9  **Protect production with an environment**
Use GitHub environments with required reviewers.
- 10  **Deploy using Helm**
Deploy your application to Kubernetes using Helm.
- 11  **Wait for Kubernetes rollout**
Verify the deployment is successful.
- 12  **Introduce a deliberately broken release**
Make a change that causes a failure.
- 13  **Diagnose the failed workflow**
Check logs, identify the issue, and fix it.
- 14  **Inspect application state**
Check the status in Kubernetes.
- 15  **Roll back**
Revert to the previous working version.
- 16  **Verify recovery**
Confirm the application is working again.
- 17  **Review workflow history**
Look at the full run history and what you learned.

2 FINAL ARCHITECTURE



3 LEARNING OUTCOMES

- ✓ Build a real CI/CD pipeline
- ✓ Work with pull requests
- ✓ Run automated tests
- ✓ Build and push container images
- ✓ Use secure authentication (OIDC)
- ✓ Deploy to Kubernetes with Helm
- ✓ Use protected environments
- ✓ Troubleshoot and roll back
- ✓ Gain confidence with production workflows

4 COMPLETE LEARNING PATH



JUNIOR ENGINEER TIP

- Don't just follow the steps. Understand what is happening at each stage.
- Break things on purpose, so you know how to fix them.
- A successful engineer can deploy, but a great engineer can also recover.

PRACTICE
TROUBLESHOOT
IMPROVE
BE PRODUCTION READY



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN • BUILD • AUTOMATE • DEPLOY • GROW

Same Skills
Bigger
Opportunities